**Spring Boot Library Management System Report ## 1. Entity Relationship Design The application manages two entities: Author and Book. - Relationship: One-to-Many. An `Author` can have multiple `Book`s, but a `Book` belongs to a single `Author`. - Author Attributes: `id` (Primary Key), `name`, `email` (Unique). - Book Attributes: `id` (Primary Key), `title`, `isbn` (Unique), `author_id` (Foreign Key to Author). Using JPA, this relationship is modeled with `@OneToMany` on the `Author` entity and `@ManyToOne` on the `Book` entity. ## 2. Implementation Details ### Database Population We used Spring Boot's SQL initialization feature**

(`spring.sql.init.mode=always`) along with a `data.sql` file placed in `src/main/resources`. Upon application startup, this script automatically executes and populates the in-memory H2 database with 10 sample authors and 10 sample books. ### Create Operation Forms: Developed JSP pages (`author-form.jsp` and `book-form.jsp`) using Spring's `form:form` tags to easily bind form inputs to model attributes. Controllers: `AppController` handles `POST` requests to `/authors/save` and `/books/save`. We use the `@Valid` annotation to enforce validation (e.g., `@NotBlank` and `@Email`). Integrity Violation: If a user tries to add an Author with an existing email, or a Book with an existing ISBN, JPA throws a `DataIntegrityViolationException`.

**This exception is caught in the controller, and an appropriate error message is displayed back to the user on the form page. ### Read Operation Listing: The `/authors` and `/books` endpoints fetch all records and render them in a tabular format in JSP. Custom Query: For books, the `BookRepository` implements a custom query to optimize fetching books along with their associated authors using an Inner Join: ```java @Query(SELECT**